

RESEARCH REPORT

Toy Blockchain CLI

Gunarathne G.D.H.

9/7/2026

1 Required investigations

1.1 Tamper-evidence

Without tampering,

```
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go
ToyBlockchain CLI
Commands:
  help      - Show this message
  addtx     - Add a new transaction (addtx sender recipient amount)
  mine      - Mine all pending transactions into a new block
  balances  - Show all account balances
  validate  - Verify the integrity of the blockchain
  print     - Print the entire JSON blockchain
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go addtx Alice Bob 67
Success! Alice sent 67 to Bob (Pending in pool)
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go addtx Bob Alice 23
Success! Bob sent 23 to Alice (Pending in pool)
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go mine
Mining block 4 with 2 transactions (Difficulty: 3)...
Block mined! Nonce: 7790, Time: 9.0714ms, Hash: 0001bb52e5c2e96680f354ee96c2107fe1c220d347c04238603032daf74b50ba
Block successfully mined and saved!
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go balances
--- Account Balances ---
Alice: 31
Bob: 69
System: 0
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go validate
Chain is valid! No tampering detected.
PS D:\Intern\Projects\GolangDevProject\toyblockchain>
```

Changing the transaction value Bob send to 23 to 230 in the JSON file

```
59 {
60   "Index": 4,
61   "Timestamp": 1783574160,
62   "Transactions": [
63     {
64       "Sender": "Alice",
65       "Recipient": "Bob",
66       "Amount": 67
67     },
68     {
69       "Sender": "Bob",
70       "Recipient": "Alice",
71       "Amount": 230
72     }
73   ],
74   "Hash": "0001bb52e5c2e96680f354ee96c2107fe1c220d347c04238603032daf74b50ba"
75 }
```

```
Problems 1 Output Debug Console Terminal Ports powershell - toyblockchain + v @
Chain is valid! No tampering detected.
PS D:\Intern\Projects\GolangDevProject\toyblockchain> go run cmd/toychain/main.go validate
Chain is INVALID! Tampering detected at block 4
PS D:\Intern\Projects\GolangDevProject\toyblockchain>
```

when transaction altered the recomputation of the block hash will not match and if some how the attacker changed the hash as well the next block securely saved the block hash will capture the block has been tampered with.

1.2 Difficulty versus effort

I added the same transaction for 5 times to see the difference to mine changing the difficulty.

Assume the affect of Index, Timestamp, prev. hash will be very small.

Difficulty	Nonce	Time Taken
1	13	0s
2	768	538.8 μ s
3	7351	7.2483ms
4	88643	67.1047ms
5	1117129	789.4279ms

Time and effort grows exponentially with the difficulty increasing.

1.3 Design write-up

In the block my hashing is done by order of , Index, Timestamp, Transaction, prevHash and Nonce. The final Hash value will saved to the block after block is mined.

My validation guarantees integrity across the whole chain by enforcing 5 rules:

- Recomputation Rule (Hashes are mathematically accurate).
- Link Rule (Blocks point to the exact previous block).
- Proof-of-Work Rule (Miners actually spent CPU effort).
- Index Rule (No missing pages).
- Time Rule (No time-traveling blocks).

2 Discussion questions

1. How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy?
 - Changing Block 2 requires re-mining Block 2, which changes its hash, which breaks Block 3, requiring to re-mine Block 3, 4, 5...
2. Proof-of-work is one way to decide who may add the next block. Name at least one alternative, such as proof-of-stake or proof-of-authority, and give one advantage and one drawback versus proof-of-work in your own words.
 - Proof-of-Stake (PoS). Advantage: Uses 99% less electricity. Drawback: "Rich get richer" problem, as those with the most coins get the most block rewards.
3. List three concrete ways your toy differs from a production blockchain (think about consensus among peers, transaction signatures, Merkle trees, and finality). Pick one and sketch how you would add it.
 - No Peer-to-Peer Networking / Consensus (Longest-chain rule).
 - No Digital Signatures (Public/Private Keys).
 - No Merkle Trees for transaction hashing.
 - Digital Signatures : Users require to sign in before make an transaction so users can sign the transaction using there private key and in the "ValidateChain" fuction would verify the signature with there public key before allowing the transaction.